

K.R. MANGALAM UNIVERSITY

DSA Assignment : 1

Submitted by:

Tushar Saini

2501730053

B.TECH CSE(AI&ML) Section : A

Submitted to:

Mrs. Neetu Chauhan

1. Introduction : In this assignment, I implemented basic Data Structures and recursive algorithms using Python. The aim was to understand recursion, time and space complexity, and different algorithmic paradigms. The toolkit includes Stack ADT, Factorial, Fibonacci, Tower of Hanoi, and Recursive Binary Search. And also some discussion on their time and space complexity.

2.Stack ADT : An Abstract Data Type (ADT) is a logical representation of a data structure that defines its operations without specifying how it is implemented. A Stack ADT follows the LIFO (Last In First Out) principle, where the last element inserted is the first one removed. The main operations of a stack are push, pop, peek, is_empty, and size. These operations take $O(1)$ time because they work only at the top of the stack. Operations in stack are mainly push and pop which are applied in the starting of the index only that's why they have time complexity as $O(1)$.

3.Factorial recursive : Factorial is implemented using recursion, where the function calls itself until it reaches the base case ($0! = 1$). Each recursive call reduces the value of n by 1 until it becomes 0. Factorial is defined as $n * \text{factorial of } (n-1)$ in it each call gets reduced by 1 so it is having time complexity as $O(n)$.

4. Fibonacci :

Naive :

Naive Fibonacci is implemented using simple recursion, where each function call computes $\text{fib}(n-1)$ and $\text{fib}(n-2)$. It has a base case when n is 0 or 1. However, this approach recalculates the same values many times, which makes it inefficient for large n . The time complexity is $O(2^n)$ because the number of recursive calls grows

exponentially. The space complexity is $O(n)$ due to the recursion call stack. It is very slow for large inputs. It recalculates same value multiple time required that's why we use the another i.e. memoized way. Naive method again and again recalculating the same value become the not suitable for the large inputs .

Memoized :

Memoized Fibonacci improves the naive recursive approach by storing already calculated values in a dictionary. Before computing a value, the function checks whether it has been calculated earlier. This avoids repeated calculations and makes the algorithm much faster. The time complexity becomes $O(n)$ because each value is computed only once. The space complexity is $O(n)$ due to the recursion stack and the extra memory used for storing previous results. It is much faster than the naive one as it does not recalculate same value multiple time and has time complexity also less than naive.

5. Tower of Hanoi : Tower of Hanoi is a classic recursive problem where disks are moved from one rod to another following certain rules. The solution works by moving $n-1$ disks to an auxiliary rod, then moving the largest disk to the destination, and finally moving the $n-1$ disks again to the destination. The base case occurs when only one disk is left. The total number of moves required is $2^n - 1$. The time complexity is $O(2^n)$ because the number of moves grows exponentially in it. The space complexity is $O(n)$ because of the recursion depth. $Moves(n) = 2 \times Moves(n-1) + 1$ If we keep expanding this pattern, it becomes: $Moves(n) = 2^n - 1$

6. Recursive Binary Search : Recursive Binary Search works by checking the middle element of a sorted array and then searching

only in one half of the array. The process continues until the element is found or the search space becomes empty. Time complexity is $O(\log n)$ in average case as it can be find in middle or any other position and worst case as if the element may be at last or not there , and $O(1)$ in best case getting the element at starting only. Space complexity is $O(\log n)$ due to recursion. It follows the Divide & Conquer approach.

7. Paradigim Reflection :

1. Divide & Conquer Divide & Conquer is a technique where a problem is divided into smaller parts, each part is solved separately, and then the results are combined. In Binary Search, the array is divided into two halves again and again. Instead of searching the whole array, we only search in one half each time. This reduces the problem size quickly and makes the algorithm efficient. Real-life example : Searching a name in a dictionary by opening it in the middle and deciding whether to go left or right.

2. Dynamic Programming Dynamic Programming is used when a problem has repeated subproblems. Instead of solving the same problem again and again, we store the results and reuse them. In Memoized Fibonacci, once a Fibonacci number is calculated, it is stored in memory. So when it is needed again, the stored value is used instead of recalculating it. This improves efficiency. Real-life example : Saving progress in a game so you don't restart from the beginning is like Dynamic Programming because previous results are stored and reused..

3. Recursion Recursion is a technique where a function calls itself to solve smaller versions of the same problem. In Tower of Hanoi, to move n disks, we first move $n-1$ disks, then move the largest disk, and again move $n-1$ disks. The process continues until only one disk

remains (base case). Recursion helps break a complex problem into simpler steps.

Real-life example : Opening folders inside folders on a computer is like recursion, where the same process repeats on smaller levels until the last folder is reached.

8. Conclusion : Through this assignment, I understood how recursion works and how to analyze algorithms using time and space complexity. I also learned the importance of different paradigms like Divide & Conquer and Dynamic Programming in improving efficiency